Earn v2 1.6

Blueprint Finance

HALBORN

Summary

100% OF ALL REPORTED FINDINGS HAVE BEEN ADDRESSED

ABOUT THIS REPORT

ASSESSMENT TYPE
Assurance Assessment

DATE OF ENGAGEMENT
Jul 31, 2026 - Aug 19, 2026

SEVERITY BREAKDOWN

24	15	2	7
All Findings	Solved	Risk Accepted	Acknowledged



INTRODUCTION

`Blueprint Finance` engaged `Halborn` to perform a security assessment of their smart contracts from August 7th, 2026 to August 18th, 2026. The assessment scope was limited to the smart contracts provided to Halborn. Commit hashes and additional details are available in the Scope section of this report.

The `Blueprint Finance` codebase in scope consists of changes made to Concrete Earn V2's vault, factory, hook, timelock, and looping strategy contracts, including a new Dolomite integration with native and AssetCX self liquidation paths and an epoch based asynchronous withdrawal queue.

ASSESSMENT SUMMARY

`Halborn` was allocated 6 days for this engagement and assigned 1 full-time security engineer to conduct a comprehensive review of the smart contracts within scope. The engineer is an expert in blockchain and smart contract security, with advanced skills in penetration testing and smart contract exploitation, as well as extensive knowledge of multiple blockchain protocols.

The objectives of this assessment are to:

- Identify potential security vulnerabilities within the smart contracts.
- Verify that the smart contract functionality operates as intended.

In summary, `Halborn` identified several areas for improvement to reduce the likelihood and impact of security risks, which were partially addressed by the `Blueprint Finance` team. The main recommendations were:

- `Add a maxSlippageInWad parameter to selfLiquidateDolomiteNative() and selfLiquidateDolomiteNativeToLtv(), validate it against maxSelfLiqSlippageInWad, and enforce a minimum received amount after the Sell action by reverting when the realized debt reduction falls below debtAmount scaled by the allowed slippage.`
- `In the Dolomite flavour initializers require DOLOMITE_MARGIN.getMarketTokenAddress(COLLATERAL_MARKET_ID()) to equal COLLATERAL_TOKEN() and the borrow market to equal BORROW_TOKEN(), reverting otherwise.`
- `In PositionHelperDolomiteNative.selfLiquidateDolomiteNativeToLtv(), size the collateral to sell by calling PositionManagementHelperLib.flashWithdrawAmountFromTargetLtv() with both the premium and the slippage factor set to zero (F equal to WAD), so the returned amount jointly accounts for collateral and debt falling together and is exact for the native full sell path.`

TEST APPROACH AND METHODOLODY

`Halborn` conducted a combination of manual code review and automated security testing to balance efficiency, timeliness, practicality, and accuracy within the scope of this assessment. While manual testing is crucial for identifying flaws in logic, processes, and implementation, automated testing enhances coverage of smart contracts and quickly detects deviations from established security best practices.
The following phases and associated tools were employed throughout the term of the assessment:

- Research into the platform's architecture, purpose and use.
- Manual code review and walkthrough of smart contracts to identify any logical issues.
- Comprehensive assessment of the safety and usage of critical Solidity variables and functions within scope that could lead to arithmetic-related vulnerabilities.
- Local testing using custom scripts (`Foundry`).
- Fork testing against main networks (`Foundry`).
- Static security analysis of scoped contracts, and imported functions (`Slither`).

Risk Methodology

Every vulnerability and issue observed by Halborn is ranked based on **two sets** of **Metrics** and a **Severity Coefficient**. This system is inspired by the industry standard Common Vulnerability Scoring System.

The two **Metric sets** are: **Exploitability** and **Impact**. **Exploitability** captures the ease and technical means by which vulnerabilities can be exploited and **Impact** describes the consequences of a successful exploit.

The **Severity Coefficients** is designed to further refine the accuracy of the ranking with two factors: **Reversibility** and **Scope**. These capture the impact of the vulnerability on the environment as well as the number of users and smart contracts affected.

The final score is a value between 0-10 rounded up to 1 decimal place and 10 corresponding to the highest security risk. This provides an objective and accurate rating of the severity of security vulnerabilities in smart contracts.

The system is designed to assist in identifying and prioritizing vulnerabilities based on their level of risk to address the most critical issues in a timely manner.

4.1 EXPLOITABILITY

ATTACK ORIGIN (AO):

Captures whether the attack requires compromising a specific account.

ATTACK COST (AC):

Captures the cost of exploiting the vulnerability incurred by the attacker relative to sending a single transaction on the relevant blockchain. Includes but is not limited to financial and computational cost.

ATTACK COMPLEXITY (AX):

Describes the conditions beyond the attacker's control that must exist in order to exploit the vulnerability. Includes but is not limited to macro situation, available third-party liquidity and regulatory challenges.

METRICS:

EXPLOITABILITY METRIC (m_e)	METRIC VALUE	NUMERICAL VALUE
Attack Origin (AO)	Arbitrary (AO:A)	1
	Specific (AO:S)	0.2
Attack Cost (AC)	Low (AC:L)	1
	Medium (AC:M)	0.67
	High (AC:H)	0.33
Attack Complexity (AX)	Low (AX:L)	1
	Medium (AX:M)	0.67
	High (AX:H)	0.33

Exploitability E is calculated using the following formula:

$$E = \prod m_e$$

4.2 IMPACT

CONFIDENTIALITY (C):

Measures the impact to the confidentiality of the information resources managed by the contract due to a successfully exploited vulnerability. Confidentiality refers to limiting access to authorized users only.

INTEGRITY (I):

Measures the impact to integrity of a successfully exploited vulnerability. Integrity refers to the trustworthiness and veracity of data stored and/or processed on-chain. Integrity impact directly affecting Deposit or Yield records is excluded.

AVAILABILITY (A):

Measures the impact to the availability of the impacted component resulting from a successfully exploited vulnerability. This metric refers to smart contract features and functionality, not state. Availability impact directly affecting Deposit or Yield is excluded.

DEPOSIT (D):

Measures the impact to the deposits made to the contract by either users or owners.

YIELD (Y):

Measures the impact to the yield generated by the contract for either users or owners.

METRICS:

IMPACT METRIC (m_I)	METRIC VALUE	NUMERICAL VALUE
Confidentiality (C)	None (C:N)	0
	Low (C:L)	0.25
	Medium (C:M)	0.5
	High (C:H)	0.75
	Critical (C:C)	1
Integrity (I)	None (I:N)	0
	Low (I:L)	0.25
	Medium (I:M)	0.5
	High (I:H)	0.75
	Critical (I:C)	1
Availability (A)	None (A:N)	0
	Low (A:L)	0.25
	Medium (A:M)	0.5
	High (A:H)	0.75

IMPACT METRIC (m_I)	METRIC VALUE	NUMERICAL VALUE
	Critical (A:C)	1
Deposit (D)	None (D:N)	0
	Low (D:L)	0.25
	Medium (D:M)	0.5
	High (D:H)	0.75
	Critical (D:C)	1
Yield (Y)	None (Y:N)	0
	Low (Y:L)	0.25
	Medium (Y:M)	0.5
	High (Y:H)	0.75
	Critical (Y:C)	1

Impact I is calculated using the following formula:

$$I = max(m_I) + \frac{\sum m_I - max(m_I)}{4}$$

4.3 SEVERITY COEFFICIENT

REVERSIBILITY (R):

Describes the share of the exploited vulnerability effects that can be reversed. For upgradeable contracts, assume the contract private key is available.

SCOPE (S):

Captures whether a vulnerability in one vulnerable contract impacts resources in other contracts.

METRICS:

SEVERITY COEFFICIENT (C)	COEFFICIENT VALUE	NUMERICAL VALUE
Reversibility (r)	None (R:N)	1
	Partial (R:P)	0.5
	Full (R:F)	0.25
Scope (s)	Changed (S:C)	1.25
	Unchanged (S:U)	1

Severity Coefficient C is obtained by the following product:

$$C = rs$$

The Vulnerability Severity Score S is obtained by:

$$S = min(10, EIC * 10)$$

The score is rounded up to 1 decimal places.

Critical	High	Medium	Low	Informational
9 - 10	7 - 8.9	4.5 - 6.9	2 - 4.4	0 - 1.9

Scope

URL

(a) URL:

https://github.com/Blueprint-Finance/earn-v2-core/compare/3b7a0e40f0738d8945828fcdd56bff5f0d6a551c...401929847078460817 7d8a30c7fb7421c8

(b) Items in scope:

- src 23 files
  - interface 1 file
    - IHook.sol Solidity
  - periphery 22 files
    - factory 1 file
      - PeripheryFactory.sol Solidity
    - hooks 8 files
      - BaseHook.sol Solidity
      - BaseOwnableHook.sol Solidity
      - DepositLockHook.sol Solidity
      - DepositLockRegSHook.sol Solidity
      - DepositLockWithFeeHook.sol Solidity
      - HookContainer.sol Solidity
      - UserDepositCapHook.sol Solidity
      - WhitelistUserDepositHook.sol Solidity
    - interface 2 files
      - IDolomiteMargin.sol Solidity
      - ILiquidationModule.sol Solidity
    - lenders 1 file
      - AaveV3LenderAdapter.sol Solidity
    - lib 4 files
      - Looping 1 file
        - LoopingPrimitiveLib.sol Solidity
      - DolomiteAccountActionLib.sol Solidity
      - PositionHelperDolomiteAssetCXStorageLib.sol Solidity
      - PositionHelperDolomiteStorageLib.sol Solidity
    - positionhelper 6 files
      - PositionHelperAaveV3.sol Solidity
      - PositionHelperBase.sol Solidity
      - PositionHelperDolomiteAssetCX.sol Solidity
      - PositionHelperDolomiteBase.sol Solidity
      - PositionHelperDolomiteNative.sol Solidity
      - PositionHelperMorpho.sol Solidity

Out-of-Scope: Third party dependencies and economic attacks.

REMEDIATION COMMIT ID:

- f34da5522865e7e274e111848969723e4b86102f
- 806ff1fffcb5586f01611022875ed455f7d40b70
- a01c7993bc47e6a3efc18efdaaaa77c3a09234aa
- 1d718335c324ab686dc944c948f0d6845305f8ed
- 5d88016589cdbd57066129b07553c052488ade23
- 4b0b2aafa199b4e78a8e0a1b485f9a62665ccefe
- 32dd228f2c1a12f9c22c3aa12be5604d871273a0
- d86e729b53474a1937dd0a96570dc7e55167a9a8
- 0890774dcfe101539990e4280b371002c3062bb0
- cf9f9a4a7d249d868705332082c02a058ecff29f
- 064e6e1af5809b6cf2ad912976611d1762ad4516
- 6f35e83562f02b899a6b86ea9b5e34c52408b4a2
- 3403ac893e956331d450d22681bcd74df3cc592b
- ebd8a3e3b705f3c371a804fc1d04f3cf933a7bec
- fef265262badf15e1196098fb5fa34a554002bfb
- 28072663cdb1c86f84a2a0005e1435ac02d3c412

Out-of-Scope: New features/implementations after the remediation commit IDs.

Assessment Summary & Findings Overview

#	TITLE	SEVERITY	SCORE	STATUS
HAL-001	Unbounded equity loss due to missing slippage protection in selfLiquidateDolomiteNative	Low	3.4	SOLVED 09/01/2026
HAL-002	Clone bound to wrong market due to silently omitted Dolomite market IDs	Low	3.1	SOLVED 09/01/2026
HAL-003	Position left over leveraged due to wrong target LTV sizing	Low	2.8	SOLVED 09/01/2026
HAL-004	Non-whitelisted addresses can hold vault shares due to ungated receivers and transfers	Low	2.5	RISK ACCEPTED 09/01/2026
HAL-005	Unsafe liquidation configuration due to missing input validation in the Dolomite initializers and setters	Low	2.0	SOLVED 09/01/2026

#	TITLE	SEVERITY	SCORE	STATUS
HAL-006	Permanent admin lockout due to missing last admin guard on role revocation	Low	2.0	SOLVED 09/01/2026
HAL-007	Silent Reg-S lock bypass due to unenforced migration chain invariant	Low	2.0	RISK ACCEPTED 09/01/2026
HAL-008	Governance lockout due to unstripped clone immutable args in timelock action ID	Informational	1.9	SOLVED 09/01/2026
HAL-009	Healthy LTV configuration inconsistency due to eMode unaware max LTV read	Informational	1.7	ACKNOWLEDGED 09/01/2026
HAL-010	Per-user deposit cap bypassable due to current-value accounting in UserDepositCapHook	Informational	1.7	ACKNOWLEDGED 09/01/2026
HAL-011	Unbounded early unlock fee due to missing caller fee bound	Informational	1.7	ACKNOWLEDGED 08/14/2026
HAL-012	Self-liquidation griefing due to unguarded collateral balance subtraction	Informational	1.7	SOLVED 09/01/2026
HAL-013	Deposit blocking griefing due to lock queue flooding via arbitrary receiver	Informational	1.6	ACKNOWLEDGED 08/14/2026
HAL-014	Vault wide operation halt when the bound vault re-enables cancellation	Informational	1.3	ACKNOWLEDGED 09/01/2026
HAL-015	LTV divergence due to missing single collateral enforcement on Dolomite helpers	Informational	1.3	SOLVED 09/01/2026
HAL-016	Privilege reductions lack advance notice due to asymmetric timelock	Informational	1.0	SOLVED 09/01/2026
HAL-017	Block of hook and PMH cloning due to missing unblock functions	Informational	1.0	SOLVED 09/01/2026
HAL-018	Self-liquidation gate ignores in-flight liquidation exposure in PositionHelperDolomiteAssetCX	Informational	1.0	ACKNOWLEDGED 09/01/2026
HAL-019	Denial of service due to missing zero price guard in DolomitePositionViewLib.collateralPriceInBorrowTokensInRay	Informational	0.7	SOLVED 09/01/2026

#	TITLE	SEVERITY	SCORE	STATUS
HAL-020	Arithmetic panic at extreme LTV due to unclamped uint64 LTV handling	Informational	0.6	SOLVED 09/01/2026
HAL-021	LTV divergence due to missing decimal conversion of redemption claim collateral	Informational	0.6	SOLVED 09/01/2026
HAL-022	Owner blocked from emergency access before strategy configuration due to check ordering in modifier	Informational	0.5	ACKNOWLEDGED 09/01/2026
HAL-023	Forced self liquidation due to unchecked destination account in debt transfers	Informational	0.5	SOLVED 09/01/2026
HAL-024	Oversized position unwind due to unclamped debt amount in selfLiquidateDolomiteNative	Informational	0.5	SOLVED 09/01/2026

Findings & Tech Details

HAL-001 SOLVED

Unbounded equity loss due to missing slippage protection in selfLiquidateDolomiteNative

Low · 3.4 AO:A/AC:L/AX:M/R:N/S:U/C:N/A:N/I:N/D:M/Y:N

DESCRIPTION

The `PositionHelperDolomiteNative.selfLiquidateDolomiteNative()` and `PositionHelperDolomiteNative.selfLiquidateDolomiteNativeToLtv()` functions unwind a position by selling collateral for the borrow token through Dolomite's native `Sell` action. They compute a fixed collateral input amount from the oracle price and forward operator supplied `swapData` to the Dolomite exchange wrapper.

However, neither function accepts a `maxSlippageInWad` argument and `_selfLiquidateDolomiteNative()` performs no minimum output check after the sale. The `Sell` action uses exact input semantics: Dolomite hands the fixed collateral amount to the wrapper and credits whatever borrow tokens come back, then verifies only that the account remains solvent. It does not verify that the proceeds meet any minimum. This contrasts with the flash loan path in `PositionHelperBase._selfLiquidate()`, which validates the caller's `maxSlippageInWad` against the admin configured `maxSelfLiqSlippageInWad` ceiling and uses exact output swap semantics.

Because `canLiquidate` admits `POSITION_OPERATOR_ROLE` holders who are distinct from the position owner, and because `swapData` is caller controlled, an operator or an MEV actor can cause the position owner to sell collateral at a rate far worse than the oracle implied rate, with no on chain cap on the loss.

For example:

1. An operator calls `selfLiquidateDolomiteNative(debtAmount, swapData)` with routing that passes through a thin liquidity pool, or an MEV bot sandwiches the transaction.
2. The oracle implies the collateral sold should return `debtAmount` of borrow tokens, so `collateralAmountToSell` is set to that collateral equivalent.
3. The `Sell` executes and returns materially fewer borrow tokens than `debtAmount` because of the manipulated or illiquid route.
4. Dolomite credits the reduced proceeds and its solvency check passes, so no revert occurs.
5. The position owner has sold the full oracle computed collateral amount but repaid far less debt than intended, incurring an uncapped equity loss.

RECOMMENDATION

Add a `maxSlippageInWad` parameter to `selfLiquidateDolomiteNative()` and `selfLiquidateDolomiteNativeToLtv()`, validate it against `maxSelfLiqSlippageInWad`, and enforce a minimum received amount after the `Sell` action by reverting when the realized debt reduction falls below `debtAmount` scaled by the allowed slippage.

REMEDIATION COMMENT

Addressed in: f34da5522865e7e274e111848969723e4b86102f

SOLVED: The `Blueprint Finance` **team** solved this finding in the specified commit by computing the oracle-implied `expectedProceeds` inside `_selfLiquidateDolomiteNative` and reverting `SlippageTooHigh` when the realized `debtRepaid` falls below `expectedProceeds * (WAD - maxSelfLiquidationSlippageInWad) / WAD`, stating:

> Remediated by checking proceeds after swapping and reverting if: `if (debtRepaid < minProceeds) revert SlippageTooHigh(debtRepaid, minProceeds);`

HAL-002 SOLVED

Clone bound to wrong market due to silently omitted Dolomite market IDs

Low · 3.1 AO:A/AC:L/AX:L/R:N/S:U/C:N/A:L/I:L/D:N/Y:N

DESCRIPTION

The `PeripheryFactory.clonePMH()` function deploys a position management helper clone and appends its immutable arguments through `PeripheryFactory._getImmutableArgs()`. For a Dolomite flavour, that helper is expected to append two 32-byte market IDs, collateral and borrow, to the 62-byte header, producing a 126-byte blob that `PositionHelperDolomiteBase.COLLATERAL_MARKET_ID()` and `PositionHelperDolomiteBase.BORROW_MARKET_ID()` read at offsets 62 and 94.

However, `PeripheryFactory._getImmutableArgs()` appends the two market IDs only when both `resolveTokenInfo` static calls succeed and both return exactly 32 bytes. `PositionHelperDolomiteBase.resolveTokenInfo()` calls `DOLOMITE_MARGIN.getMarketIdByTokenAddress(token)`, which reverts for any token that is not a registered Dolomite market. When either leg reverts, the append is skipped with no error, and the clone is created with only the 62-byte header. `PeripheryFactory.clonePMH()` validates only that the three addresses are non-zero, and the Dolomite initializers re-validate only the two decimals bytes inside the header, never the market IDs, so initialization succeeds and the factory records the clone as registered.

The clone is then structurally broken. `COLLATERAL_MARKET_ID()` and `BORROW_MARKET_ID()` read past the end of the immutable-args region and return garbage rather than reverting, with the collateral market ID reading the 2-byte length suffix as `62 << 240` and the borrow market ID reading `0`. Every subsequent supply, withdraw, borrow, and repay operates against that wrong market ID while the ERC-20 transfers still use the configured token addresses, so the clone either reverts on every operation or transacts against an unrelated market. Because `PeripheryFactory.clonePMH()` is permissionless and the malformed clone is produced with no loud failure at deploy or initialize time, an integrator deploying on behalf of a user with a collateral address that is a valid ERC-20 but not a listed Dolomite market ends up with a registered but unusable helper.

RECOMMENDATION

In the Dolomite flavour initializers `PositionHelperDolomiteNative.initialize()` and `PositionHelperDolomiteAssetCX.initialize()`, assert that each configured market id round trips to its token before initialization completes: require `DOLOMITE_MARGIN.getMarketTokenAddress(COLLATERAL_MARKET_ID())` to equal `COLLATERAL_TOKEN()` and the borrow market to equal `BORROW_TOKEN()`, reverting otherwise. Bounding each market id below `DOLOMITE_MARGIN.getNumMarkets()` first keeps the failure as a descriptive revert rather than a Dolomite index error. This rejects both a clone deployed without the market id suffix and one bound to the wrong market.

REMEDIATION COMMENT

Addressed in: 806ff1fffcb5586f01611022875ed455f7d40b70

SOLVED: The `Blueprint Finance` **team** solved this finding in the specified commit by adding a round-trip market-ID assertion in `DolomiteInitLib.init` that reverts `MarketIdTokenMismatch` when a configured

market ID does not resolve back to its token.

HAL-003 SOLVED

Position left over leveraged due to wrong target LTV sizing

Low · 2.8 AO:S/AC:L/AX:L/R:N/S:C/C:N/A:N/I:C/D:M/Y:N

DESCRIPTION

The `PositionHelperDolomiteNative.selfLiquidateDolomiteNativeToLtv()` function reduces a leveraged Dolomite position to a caller supplied target LTV by selling collateral for the borrow token through a Dolomite whitelisted exchange wrapper and crediting the proceeds against the position's debt in a single `Sell` action. It sizes the debt to repay by calling `PositionManagementHelperLib.repayAmountForTargetLtv()` and then converts that figure into an amount of collateral to sell at the oracle price.

However, `selfLiquidateDolomiteNativeToLtv()` passes a withdrawn collateral argument of zero to `repayAmountForTargetLtv()`, which solves the target LTV equation holding the collateral balance constant. That formula is only correct when the repayment is funded from outside the position, as on the plain repay path. On the native sell path the repayment is funded by selling collateral, so the collateral balance also shrinks by the sold amount. Because the sizing ignores this reduction, the function repays far less than the amount required to reach the requested LTV, and the position ends materially more leveraged than requested.

For a position holding 100 units of collateral against 80 units of debt at a 1:1 oracle price, a request to reach 50% LTV sizes the repayment at 30 and lands the position at roughly 71% LTV instead of 50%. Reaching 50% would require repaying about 60. An operator or admin who calls this function to move a position out of the self liquidation buffer zone is left with a position still close to the liquidation threshold, which third parties can then liquidate at a discount, transferring value away from the vault. Repeated calls converge on the target only asymptotically, each incurring additional swap cost.

RECOMMENDATION

In `PositionHelperDolomiteNative.selfLiquidateDolomiteNativeToLtv()`, size the collateral to sell by calling `PositionManagementHelperLib.flashWithdrawAmountFromTargetLtv()` with both the premium and the slippage factor set to zero (F equal to WAD), so the returned amount jointly accounts for collateral and debt falling together and is exact for the native full sell path. Use that return value directly as `collateralAmountToSell`, and keep the existing `TargetLtvTooHigh` zero return check and the max sentinel guard.

REMEDIATION COMMENT

Addressed in: a01c7993bc47e6a3efc18efdaaaa77c3a09234aa

SOLVED: The `Blueprint Finance` **team** solved this finding in the specified commit by replacing the incorrect `repayAmountForTargetLtv` sizing with a new `PMHLib.sellAmountForTargetLtv`, which jointly solves for collateral and debt falling together on the native sell path.

HAL-004 RISK ACCEPTED

Non-whitelisted addresses can hold vault shares due to ungated receivers and transfers

Low · 2.5 AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:L/D:N/Y:N

DESCRIPTION

The `WhitelistHook` and `WhitelistUserDepositHook` contracts enforce a KYC style allowlist on vault participation, reverting deposits and mints tied to accounts that are not on the whitelist. Their stated purpose is to ensure that only approved addresses hold exposure to the vault.

However, both hooks gate only the initiation of a deposit and never constrain who ultimately holds the minted shares. The ERC-4626 `deposit()` and `mint()` entry points let the caller name any receiver, and vault shares are a standard transferable ERC-20, so a whitelisted account can direct freshly minted shares to a non whitelisted address or move existing shares there with a plain transfer. Neither hook enforces the whitelist on share transfers, so the allowlist restricts entry through the deposit and merchant flows but not who may hold a position in the vault.

Instances of this issue include:

- `WhitelistUserDepositHook.preDeposit()` and `WhitelistUserDepositHook.preMint()`: validate only the sender initiating the deposit and never the receiver of the shares, and `preTransfer` is never overridden so it remains a no-op, leaving shares freely transferable to non whitelisted addresses.
- `WhitelistHook.preDeposit()` and `WhitelistHook.preMint()`: gate the recipient on deposit and mint, but the hook never overrides `preTransfer()` or `postTransfer()` and inherits the no-op `BaseHook` defaults, so a whitelisted holder can transfer shares to any non whitelisted address. The merchant itself moves shares with `safeTransferFrom` during account migration, so nothing forces a share transfer through a whitelist check.

RECOMMENDATION

Enforce the whitelist on the share recipient rather than only the deposit initiator: add a `whitelist[receiver]` check alongside the existing `whitelist[sender]` check in `WhitelistUserDepositHook.preDeposit()` and `WhitelistUserDepositHook.preMint()`, and in both `WhitelistHook` and `WhitelistUserDepositHook` override `preTransfer()` to require that the recipient is whitelisted while enabling the `PRE_TRANSFER` flag in the vault's hook bitmask so the override runs.

Alternatively, document explicitly that the whitelist gates only the initiator of a deposit and not the recipient or subsequent holders of the shares.

REMEDIATION COMMENT

Addressed in: `1d718335c324ab686dc944c948f0d6845305f8ed`

RISK ACCEPTED: The `Blueprint Finance` **team** made a business decision to accept the risk of this finding and not alter the contracts, clarifying the intended behavior in NatSpec, stating:

> This was a design choice. The whitelist gates only deposit and mint initiation (sender), not the share receiver or subsequent transfers. We updated the NatSpec to reflect a more precise description of how this works.

HAL-005 SOLVED

Unsafe liquidation configuration due to missing input validation in the Dolomite initializers and setters

Low · 2.0 AO:S/AC:L/AX:L/R:N/S:U/C:N/A:C/I:N/D:N/Y:N

DESCRIPTION

The `PositionHelperDolomiteNative.initialize()`, `PositionHelperDolomiteAssetCX.initialize()`, and `DolomiteLiquidationModule.initialize()` functions, together with the `DolomiteLiquidationModule.setCollateralRatio()` and `DolomiteLiquidationModule.setBuffer()` setters, assign configuration values that govern the self liquidation trigger and the collateral safety margin used during Dolomite liquidations. These values are expected to sit within safe ranges and to reference valid token addresses so that liquidations keep positions adequately backed.

However, several of these assignments write to state without validating the input, and the missing checks are as follows:

- In `PositionHelperDolomiteNative.initialize()` and `PositionHelperDolomiteAssetCX.initialize()`, the healthy LTV band, deviation, and self liquidation buffer are set without calling `_ltvRangeIsValid()`, so their sum can reach or exceed the lender liquidation threshold. The sibling Aave and Morpho initializers and the post deployment setters all call this guard, so the Dolomite initializers are the only path that skips it. The result is an operator self liquidation trigger that either fires at all times or never fires.
- In `DolomiteLiquidationModule.initialize()` and `DolomiteLiquidationModule.setCollateralRatio()`, `collateralRatioWad` is validated only with `collateralRatioWad > WAD`, so `WAD` itself (100% LTV) is accepted. The mint formula then sizes the temporary position collateral equal to the debt value with zero margin, so any accrued borrow interest pushes the position below full backing.
- In `DolomiteLiquidationModule.initialize()` and `DolomiteLiquidationModule.setBuffer()`, `bufferWad` is validated only against an upper bound of `WAD`, so it can be set to zero. With a zero buffer the extracted collateral is sized at exactly the oracle value of the moved debt, leaving no margin to cover settlement window interest and fees.
- In `DolomiteLiquidationModule.initialize()`, the collateral token derived from `IConcreteMerchant.assetCX()` is stored without a zero address check. A merchant proxy that has not yet been initialized returns `address(0)`, so the module initializes cleanly with `collateralToken == address(0)` and the misconfiguration surfaces only later when liquidation operations perform ERC20 calls against the zero address.

The correctability of these misconfigurations differs by instance. The first three are recoverable on chain through existing admin and timelock guarded setters: the LTV band values can be repaired via `setHealthyLtvInWad()`, `setHealthyLtvDeviationInWad()`, and `setSelfLiqThrBufferInWad()`, `collateralRatioWad` via `setCollateralRatio()`, and `bufferWad` via `setBuffer()`, all without redeployment. The fourth instance is different: the `assetCX()` derived collateral token has no setter, and because the module is deployed behind a UUPS proxy, correcting a stored zero address collateral token genuinely requires a contract upgrade.

RECOMMENDATION

Add input validation before the affected assignments, noting the two contracts involved are distinct. In the position helper initializers `PositionHelperDolomiteNative.initialize()` and `PositionHelperDolomiteAssetCX.initialize()`, call the inherited `_ltvRangeIsValid()` guard that the Aave and Morpho initializers already use. In `DolomiteLiquidationModule.initialize()` and `DolomiteLiquidationModule.setCollateralRatio()`, reject `collateralRatioWad` values at or above a safe maximum strictly below `WAD`; in `DolomiteLiquidationModule.initialize()` and `DolomiteLiquidationModule.setBuffer()`, enforce a minimum floor on `bufferWad` sized to cover settlement window interest and fees; and in `DolomiteLiquidationModule.initialize()`, revert when `IConcreteMerchant.assetCX()` returns the zero address.

REMEDIATION COMMENT

Addressed in: 5d88016589cdbd57066129b07553c052488ade23

SOLVED: The `Blueprint Finance` **team** solved this finding in the specified commit by adding the `_ltvRangeIsValid()` guard to both Dolomite position-helper initializers and tightening the `DolomiteLiquidationModule` initializer and setters (reject `collateralRatioWad >= WAD`, reject zero `bufferWad` / `liquidatableLtvWad`, and revert when `assetCX()` is the zero address), stating:

> Fixed missing validation in initializer regarding ltv range. also added clamping the debt repayment in the self-liquidate to the available debt at that current moment. The remaining fixes relate to another repository and there is a remediation pr up on that side.

HAL-006 SOLVED

Permanent admin lockout due to missing last admin guard on role revocation

Low · 2.0 AO:S/AC:L/AX:L/R:N/S:U/C:N/A:C/I:N/D:N/Y:N

DESCRIPTION

The `ConcreteMerchant` and `AssetCX` contracts inherit `AccessControlUpgradeable` and gate their most sensitive operations behind `DEFAULT_ADMIN_ROLE`. These operations include `_authorizeUpgrade()` on both UUPS proxies, `setTimelock()`, `activateTimelock()`, `emergencyMigrateUserAccount()`, and subordinate role management on `ConcreteMerchant`, and pauser administration on `AssetCX`. Because `DEFAULT_ADMIN_ROLE` is its own role admin, only an account that already holds it can grant it to anyone else.

However, an earlier baseline used `AccessControlDefaultAdminRulesUpgradeable`, which structurally prevents the default admin set from being emptied by enforcing a delayed two step ownership transfer. Both contracts now use plain `AccessControlUpgradeable` with a single `_grantRole(DEFAULT_ADMIN_ROLE, admin)` at initialization and no override of `_revokeRole()` to keep at least one admin present. Both `revokeRole()` and `renounceRole()` route through `_revokeRole()`, so the sole admin can renounce, or be revoked while last, in a single transaction that leaves the admin set empty.

Once the admin set is empty, no account can grant `DEFAULT_ADMIN_ROLE` again. On both UUPS proxies `_authorizeUpgrade()` reverts, so neither contract can be upgraded to restore control, and every admin gated function on both contracts, namely upgrades, timelock configuration, emergency account migration, subordinate role management, and pauser administration, becomes permanently unreachable.

RECOMMENDATION

In `ConcreteMerchant` and `AssetCX`, override `_grantRole()` and `_revokeRole()` to maintain a dedicated count of `DEFAULT_ADMIN_ROLE` holders and keep at least one holder at all times, initializing that count inside a post upgrade reinitializer that tallies the existing admins under a version not already consumed by these contracts, such as `reinitializer(3)`. Additionally, declare a `CannotRemoveLastRoleAdmin` error in `IAssetCX`, mirroring the last admin guard already implemented in `DolomiteLiquidationModule._revokeRole()`.

REMEDIATION COMMENT

Addressed in: 4b0b2aafa199b4e78a8e0a1b485f9a62665ccefe

SOLVED: The `Blueprint Finance` **team** solved this finding in the specified commit by switching `AssetCX` and `ConcreteMerchant` to `AccessControlEnumerableUpgradeable`, overriding `_revokeRole` to revert `CannotRemoveLastRoleAdmin` when the final `DEFAULT_ADMIN_ROLE` holder would be removed, and adding a reinitializer that backfills the enumerable admin set.

HAL-007 RISK ACCEPTED

Silent Reg-S lock bypass due to unenforced migration chain invariant

Low · 2.0 AO:S/AC:L/AX:L/R:N/S:U/C:N/A:N/I:C/D:N/Y:N

DESCRIPTION

The `DepositLockRegSHook` contract implements a hook migration chain so that when a vault swaps one lock hook for a newer one, the successor keeps enforcing the predecessor's still active Regulation S transfer and redeem locks. It links hooks through `legacyHook` (the immediate predecessor, set in `initialize` and downgradeable to zero via `disconnectLegacyHook`) and `successorHook` (set via `setSuccessorHook`). On every enforcement path, `DepositLockRegSHook.preTransfer()` routes through `_requireTransferUnlocked()` and `DepositLockRegSHook.preWithdraw()` / `preRedeem()` route through `_enforceRedeemAndConsumeTransferLock()`, and both call `_legacyDrain()`, which invokes `legacyHook.drainAndGetLocked()` on the direct predecessor to fold the predecessor's locked shares into the check.

The contract documents a single hop migration invariant: before wiring a new successor, the immediate predecessor must have fully drained its locks and had `disconnectLegacyHook()` called so its own `legacyHook` reads as `address(0)`. However, `DepositLockRegSHook.drainAndGetLocked()` returns only this hook's own `_totalLocked[user]` and `_effectiveRedeemLocked(user)`. It never calls `_legacyDrain()`, so it does not recurse into its own `legacyHook`. Neither `DepositLockRegSHook.initialize()` nor `DepositLockRegSHook.setSuccessorHook()` requires that the predecessor being wired has `legacyHook == address(0)`, so nothing on chain enforces the single hop invariant and nothing prevents wiring a third hook while the second still points at the first. In a three hook chain v1, v2, v3 where `v3.legacyHook == v2` and `v2.legacyHook == v1`, the enforcing hook v3 drains v2 through `drainAndGetLocked()`, which drops v1's still active locks entirely.

The result is a silent bypass of the Reg-S transfer and redeem lock for any user who still holds shares locked on v1 at the time of the second migration. If the operator wires the chain v1 to v2 to v3 without calling `disconnectLegacyHook()` on v2 and before v1's locks expire, `v3.getUnlockedTransferShares` reports the v1 hard locked shares as free, so those shares become freely transferable and redeemable with no revert and no event, defeating the transfer restriction the hook exists to enforce. Reaching this state requires the operator to deviate from the documented migration procedure, since a correctly sequenced migration drains and disconnects each predecessor before wiring the next.

RECOMMENDATION

In `DepositLockRegSHook.initialize()`, add `legacyHook()` to the `IDepositLockRegSHook` interface and, whenever a nonzero predecessor is supplied, require `IDepositLockRegSHook(legacyHook_).legacyHook()` to read `address(0)` before wiring it while retaining the existing same vault validation, so only a fully unchained predecessor can be linked and genesis deployments that supply no predecessor still proceed.

Additionally, in `DepositLockRegSHook.setSuccessorHook()`, require the current hook's own `legacyHook` to be `address(0)` before accepting any nonzero successor, so a hook that still points at an earlier predecessor cannot itself become a link in a longer chain. Keep

`DepositLockRegSHook.disconnectLegacyHook()` as an unconditional owner-only detach so a reverting or bricked predecessor can always be removed and successor transfers and redeems stay live.

REMEDIATION COMMENT

RISK ACCEPTED: The `Blueprint Finance` **team** made a business decision to accept the risk of this finding and not alter the contracts, stating:

> This finding was discussed in a previous audit. Legacy aggregation is single-hop by design. Multi-step migration requires the predecessor to be empty and disconnected before the next hook is wired. Spec §6.5 already documents this; nested chains are unsupported.

HAL-008 SOLVED

Governance lockout due to unstripped clone immutable args in timelock action ID

Informational · 1.9 AO:A/AC:L/AX:L/R:F/S:U/C:N/A:H/I:N/D:N/Y:N

DESCRIPTION

The `HookContainer._enforceDelay()` function implements the timelock gate that protects every privileged operation on the container: `addHook()`, `replaceHook()`, `removeHook()`, `setTimelock()`, `grantRole()`, and `_authorizeUpgrade()` all pass through the `afterDelay` modifier, which calls `TimelockIntegrationLib.enforceDelay(msg.sig, msg.data[4:])`. That forwards the selector and the argument bytes to `ITimelock.checkDelayExpired(sig, args)`. The `Timelock` computes the proposal action ID as `keccak256(abi.encode(target, selector, args))` and reverts with `UnknownAction` unless a matured proposal produces the same ID, so a proposer registers the proposal against the plain calldata of the intended call and the args at execution time must equal the args that were registered.

However, `HookContainer` is deployed as a `BaseHook` clone via `PeripheryFactory.cloneHook()`, which appends a 40-byte `[factory (20)][vault (20)]` immutable args tail, plus a 2-byte length suffix, to the clone's calldata under the clones with immutable args pattern that `FACTORY()` and `VAULT()` read from. Because `_enforceDelay()` slices `msg.data[4:]` without stripping that tail, the `args` passed to `checkDelayExpired()` at execution always carry the extra bytes. The `Timelock` does not strip or normalize trailing calldata, so the recomputed action ID can never equal the one a proposer computed over the plain arguments.

Once `activateTimelock()` is called, which is an irreversible and intended production step, every gated call reverts because the action ID that includes the tail has no matured proposal. Through the normal proposal flow the container's entire administrative surface becomes unusable: hooks cannot be added, removed, or replaced, roles cannot be granted, the timelock address cannot be changed, and the implementation cannot be upgraded, since the UUPS `_authorizeUpgrade` is itself `afterDelay`. The vault's hook set is frozen, so a registered hook that later needs removal (the contract's own documentation notes a malicious or buggy hook can block all vault operations) cannot be removed through governance. Recovery exists only through a non-obvious path: a proposer who manually appends the same 40-byte immutable args tail and 2-byte length suffix to the proposed arguments produces an action ID that matches at execution, but no standard proposal tooling constructs that input, so ordinary governance actions revert indefinitely.

RECOMMENDATION

In `HookContainer._enforceDelay()`, and any other `BaseHook` derived clone that uses the timelock, pass `msg.data[4:_getImmutableArgsOffset()]` to `TimelockIntegrationLib.enforceDelay()` so the appended immutable args tail is stripped and the recomputed action ID matches the one registered at proposal time.

REMEDIATION COMMENT

Addressed in: 32dd228f2c1a12f9c22c3aa12be5604d871273a0

SOLVED: The `Blueprint Finance` **team** solved this finding in the specified commit by slicing `msg.data[4:_getImmutableArgsOffset()]` in `HookContainer._enforceDelay`, stripping the clone's immutable-args tail before the action ID is recomputed.

HAL-009 ACKNOWLEDGED

Healthy LTV configuration inconsistency due to eMode unaware max LTV read

● Informational · 1.7 AO:A/AC:L/AX:M/R:N/S:U/C:N/A:L/I:N/D:N/Y:N

DESCRIPTION

The `PositionHelperAaveV3.getMaxLtvInWad()` function returns the maximum loan to value that bounds the target LTV accepted by `deploy()`. It reads the base reserve max LTV via `DATA_PROVIDER.getReserveConfigurationData(COLLATERAL_TOKEN())`, which ignores any Aave eMode category the position owner has enabled.

However, `getLiquidationThresholdInWad()` reads the account wide liquidation threshold via `POOL.getUserAccountData(POSITION_OWNER())`, which reflects the higher eMode threshold when eMode is active. `_ltvRangeIsValid()` validates the configured healthy LTV against this eMode aware liquidation threshold, while `deploy()` rejects any target LTV above the base reserve max LTV. When the position owner is in eMode, an account holding `POSITION_ADMIN_ROLE` can configure a healthy LTV that passes validation against the high eMode threshold yet sits above the base reserve max LTV.

Because `deploy()` accepts a free `targetLtvInWad` argument that is not bound to `healthyLtvInWad`, any deploy targeting a value at or below the base reserve max LTV still succeeds, and `POSITION_ADMIN_ROLE` can lower `healthyLtvInWad` on chain at any time. The effect is therefore a self correctable configuration inconsistency: a `deploy()` call aimed at the configured healthy LTV reverts with `TargetLtvExceedsMaxLtv`, leaving that specific target unreachable until the configuration is adjusted, rather than a position that is permanently stuck.

RECOMMENDATION

Make `PositionHelperAaveV3.getMaxLtvInWad()` read the LTV (the fifth return value) from the same `POOL.getUserAccountData(POSITION_OWNER())` call that `getLiquidationThresholdInWad()` already uses, applying the identical empty position fallback to the base reserve LTV. This account level read is already eMode aware and stays at or below the liquidation threshold in every state, including an empty position and an eMode category that excludes the collateral token, so the deploy bound tracks validation and never admits an immediately liquidatable position.

REMEDIATION COMMENT

ACKNOWLEDGED: The `Blueprint Finance` **team** made a business decision to acknowledge this finding and not alter the contracts, stating:

> We don't use eMode on these positions, nothing in the contract turns it on, and we don't enable it during setup, so the situation the finding describes doesn't come up. And if it ever did, the base reserve max LTV is always below the eMode liquidation threshold, so the worst it does is block borrowing that Aave would have allowed. It can never let the position open above the liquidation line. No funds at risk.

HAL-010 ACKNOWLEDGED

Per-user deposit cap bypassable due to current-value accounting in UserDepositCapHook

• Informational · 1.7 AO:A/AC:L/AX:M/R:N/S:U/C:N/A:L/I:N/D:N/Y:N

DESCRIPTION

The `UserDepositCapHook._checkDepositLimit()` function, invoked from `UserDepositCapHook.preDeposit()` and `UserDepositCapHook.preMint()`, enforces a per user deposit cap for a vault. Its documentation states that it tracks cumulative deposits per user, and it computes the checked amount as `IERC4626(VAULT()).convertToAssets(balanceOf(receiver))` plus the incoming deposit, comparing the result against `userDepositCap`.

However, this measures the current asset value of the receiver's share balance rather than a cumulative deposited total. Because share value moves with the vault exchange rate and the vault shares are a freely transferable ERC-20 that the hook does not gate on transfer, the measured amount can fall below what the receiver actually deposited, reopening headroom under the cap. Two conditions produce this: after a vault loss `convertToAssets(userShares)` decreases, so an account already at the cap gains fresh headroom; and an account at the cap can transfer its shares to another address it controls, resetting the measured balance to zero before depositing again.

The result is that a single address can accumulate exposure above its own configured cap. The effect is bounded: the per address cap is a soft, best effort limit that any party can already circumvent by spreading deposits across fresh receiver addresses, so the accounting flaw widens an existing gap rather than introducing a new bypass. The current value accounting and the absence of a share transfer gate are pre-existing behavior that sits outside the scope of the current changes.

RECOMMENDATION

Track a per user cumulative deposited amount in a storage mapping on `UserDepositCapHook`, and enforce the cap against that stored total in `UserDepositCapHook.preDeposit()` and `UserDepositCapHook.preMint()`, incrementing it on each deposit. Since this makes both functions state changing, also drop the `view` modifier from the `preDeposit()` and `preMint()` overrides in `WhitelistUserDepositHook`, which call `super`, and keep `canUserDeposit()` as a separate view helper. The cap stays a soft limit that a single party can still exceed by spreading deposits across multiple addresses.

Alternatively, if bounding only current position value is intended, document that vault losses and share transfers out of an account free deposit headroom.

REMEDIATION COMMENT

Addressed in: d86e729b53474a1937dd0a96570dc7e55167a9a8

ACKNOWLEDGED: The `Blueprint Finance` **team** made a business decision to acknowledge this finding and not alter the contracts, clarifying the intended behavior in NatSpec, stating:

> This was a design choice. We updated the NatSpec to reflect a more precise description of how this works: a soft per-receiver cap on current position value (`convertToAssets(balanceOf(receiver)) + incoming assets`), not cumulative deposits.

HAL-011 ACKNOWLEDGED

Unbounded early unlock fee due to missing caller fee bound

• Informational · 1.7 AO:A/AC:L/AX:M/R:N/S:U/C:N/A:N/I:N/D:L/Y:N

DESCRIPTION

The `DepositLockWithFeeHook.earlyUnlock()` function lets a user release locked shares before expiry by paying a time decaying fee in vault shares, computed in `DepositLockWithFeeHook._consumeLocks()`. The per lock duration is snapshotted at deposit time, and the fee is charged against the shares the user is releasing.

However, the fee rate `maxEarlyUnlockFee` is read live from storage at unlock time, `DepositLockWithFeeHook.earlyUnlock()` accepts no caller supplied maximum fee or minimum received argument, and `DepositLockWithFeeHook.setMaxEarlyUnlockFee()` is a plain `onlyOwner` setter with no timelock that accepts any value up to `FEE_BASIS` (100%). Because `_totalLocked[user]` is decremented before the fee `safeTransferFrom` (CEI ordering), the vault's own `preTransfer` enforcement passes even when the fee equals the full amount being unlocked. A hook owner can therefore set `setMaxEarlyUnlockFee(FEE_BASIS)` so that a pending or in flight `earlyUnlock()` is charged up to 100% of the shares released, and any honest fee change also lands on in flight unlocks.

The full fee vector requires a malicious or compromised `onlyOwner` actor rather than an unprivileged caller, and the resulting harm is bounded to that user's own released shares. The `setMaxEarlyUnlockFee()` setter, the live fee read in `_consumeLocks()`, and the `earlyUnlock()` signature are all unmodified by this change, which only rewrote `address(VAULT)` to `VAULT()` in this file, so the behavior is preexisting rather than introduced here.

RECOMMENDATION

Add a caller supplied `maxFeeShares` argument to `DepositLockWithFeeHook.earlyUnlock()` that reverts when the computed fee exceeds it, which fully neutralizes the front run and in flight fee inflation vector. Additionally, add a timelock to `DepositLockWithFeeHook.setMaxEarlyUnlockFee()`.

REMEDIATION COMMENT

ACKNOWLEDGED: The Blueprint **team** made a business decision to acknowledge this finding and not alter the contracts, stating:

> This was a design choice. maxEarlyUnlockFee is a live owner parameter (capped at 100% of the shares the caller chooses to unlock, then time-decayed). earlyUnlock has no slippage bound; the hook owner is trusted. CEI (_totalLocked before transferFrom) is required so the fee transfer is not blocked by the lock itself. No logic change

HAL-012 SOLVED

Self-liquidation griefing due to unguarded collateral balance subtraction

Informational · 1.7 AO:A/AC:H/AX:M/R:N/S:U/C:N/A:H/I:N/D:N/Y:N

DESCRIPTION

The `PositionHelperBase.executeFlash()` function is the flash loan callback that performs self liquidation for the flash based position helpers (the Aave V3 and Morpho variants). It runs four steps: repay the position debt with the flash borrowed tokens, withdraw `collateralToWithdraw` collateral from the lender, swap that collateral to exactly the amount owed to the flash provider through `swapExactAmountOut()`, and re-supply whatever collateral the swap did not consume. To size that re-supply it derives the consumed collateral from the contract's live token balance, setting `surplusCollateral = IERC20(COLLATERAL_TOKEN()).balanceOf(address(this))` and then `collateralUsed = collateralToWithdraw - surplusCollateral`.

However, this computation assumes the helper holds only the swap surplus at that moment, so the full balance can never exceed `collateralToWithdraw`. The collateral token is an ordinary ERC-20 and any account can transfer tokens directly to the helper. After a donation of D, the post swap balance becomes `D + collateralToWithdraw - used`, where `used` is the collateral the exact out swap actually consumed. Because `collateralToWithdraw` is inflated by the slippage buffer while the swap only consumes the true collateral value of the repaid debt, `used` is strictly below `collateralToWithdraw`. The subtraction `collateralToWithdraw - surplusCollateral` then evaluates to `used - D`, which underflows under Solidity 0.8 checked arithmetic whenever D exceeds `used`, reverting the entire self liquidation.

The donation required is not dust: it must exceed `used`, which is approximately the full collateral value of the debt slice being self liquidated, so any external account must commit a material amount of collateral, gaining nothing itself, to block self liquidation of a position. While the block holds, the position can be liquidated at the external lender, causing a loss to the position owner. The donated tokens are not extracted by the attacker; they remain in the helper, and a `POSITION_ADMIN_ROLE` holder can recover them through `rescueFunds()` and then retry the self liquidation, so the block is fully reversible and sustaining it through repeated re-donation is economically self defeating. In addition, the affected `surplusCollateral` and `collateralUsed` logic in `PositionHelperBase.executeFlash()` is byte identical to the reviewed baseline and lies outside the in-scope diff.

RECOMMENDATION

In `PositionHelperBase.executeFlash()`, stop deriving the consumed collateral from the full contract balance. Snapshot the collateral balance immediately before the withdrawal and compute the consumed amount as `balanceBefore + collateralToWithdraw - balanceAfter`. Alternatively, cap the surplus at `collateralToWithdraw` so a direct token donation cannot underflow the subtraction.

REMEDIATION COMMENT

Addressed in: 0890774dcfe101539990e4280b371002c3062bb0

SOLVED: The `Blueprint Finance` **team** solved this finding in the specified commit by snapshotting the collateral balance before the withdrawal and deriving the surplus as `balanceAfter - balanceBefore`, so a direct token donation cannot underflow the consumed-collateral subtraction.

HAL-013 ACKNOWLEDGED

Deposit blocking griefing due to lock queue flooding via arbitrary receiver

● Informational · 1.6 AO:A/AC:L/AX:H/R:N/S:U/C:N/A:M/I:N/D:N/Y:N

DESCRIPTION

The `DepositLockHook.postDeposit()` and `DepositLockHook.postMint()` functions call `_appendLock(receiver, shares)` using the ERC-4626 `receiver` address rather than the depositor, so a lock entry is created for whoever is named as the receiver of a deposit. The `_appendLock()` function reverts with `MaxLocksExceeded` once a user already holds `maxLocksPerUser` active locks (default 10, ceiling 500).

However, ERC-4626 allows any account to deposit on behalf of an arbitrary receiver. Any account can therefore call `vault.deposit(minAmount, victim)` up to `maxLocksPerUser` times, each call creating a fresh non expired lock entry for the victim and filling the victim's lock queue.

Once the queue is full, the victim cannot receive any further deposit to that address until at least one lock expires, which can be as long as `depositLockDuration`. The attacker funds these deposits and the resulting shares mint to the victim, so the attacker gifts value to the victim rather than retaining shares. The cost to the attacker is the gifted minimum deposits plus gas, and the victim can still deposit from a different address, which bounds the impact. The base contract comment acknowledges this on behalf of griefing surface but adds no per sender or on behalf of cap.

RECOMMENDATION

In `DepositLockHook`, apply a separate, lower cap for locks created when `sender != receiver` (on behalf of deposits). This requires changing `DepositLockHook.postDeposit()` and `DepositLockHook.postMint()` to plumb `sender` through to `_appendLock()`, since `_appendLock()` is not passed `sender` today, so the fix cannot live inside `_appendLock()` alone. Alternatively, merge locks created for the same receiver within the same block into a single entry so a third party cannot inflate the queue count.

REMEDIATION COMMENT

ACKNOWLEDGED: The Blueprint **team** made a business decision to acknowledge this finding and not alter the contracts.

HAL-014 ACKNOWLEDGED

Vault wide operation halt when the bound vault re-enables cancellation

Informational · 1.3 AO:A/AC:L/AX:L/R:P/S:U/C:N/A:L/I:N/D:N/Y:N

DESCRIPTION

The `DepositLockRegSHook._requireCancellationDisabled()` function enforces the hook's Regulation S guarantee that a user cannot escape the transfer lock by requesting a redeem (which consumes the Reg-S transfer lock) and then cancelling that request to recover the now freed shares. It reads `IVaultCancellationConfig(VAULT()).cancellationEnabled()` live and reverts with `CancellationEnabled` when the bound vault still allows cancelling pending withdrawal requests. It is called on every enforcement path: `DepositLockRegSHook.preTransfer()` (via `_requireTransferUnlocked()`) and `DepositLockRegSHook.preWithdraw()` / `DepositLockRegSHook.preRedeem()` (via `_enforceRedeemAndConsumeTransferLock()`).

However, the guard is re-evaluated on every single call, and the hook stores no flag recording that cancellation was already verified disabled at `initialize()` time. On async vaults, `cancellationEnabled()` is a mutable property the vault admin can flip back on at any time through the vault's `configure()` entrypoint. Nothing on the hook prevents this, and the hook exposes no owner recovery function.

If the vault admin re-enables cancellation after the hook is deployed, every `preTransfer`, `preWithdraw`, and `preRedeem` call reverts. This blocks all users from transferring shares (including fully unlocked shares and shares that never carried a Reg-S restriction), withdrawing, or redeeming, until cancellation is disabled again. Because `postDeposit` and `postMint` do not call the guard, deposits continue to succeed and new locks keep accumulating that can never be consumed while the halt is in effect. The scope of the block is broader than the request then cancel bypass the guard targets, since it also freezes already unlocked shares and legitimately redeemable Reg-S shares.

RECOMMENDATION

In `DepositLockRegSHook`, scope the cancellation guard to only the redeem request consumption path that the request then cancel bypass actually exercises, rather than reverting on every transfer, withdraw, and redeem.

Alternatively, add a hook owner recovery mechanism so the halt can be lifted at the hook level, and at minimum document that the vault admin must never re-enable cancellation while a `DepositLockRegSHook` is attached.

REMEDIATION COMMENT

Addressed in: cf9f9a4a7d249d868705332082c02a058ecff29f

ACKNOWLEDGED: The `Blueprint Finance` **team** made a business decision to acknowledge this finding and not alter the contracts, clarifying the intended behavior in NatSpec, stating:

> We don't consider this a low finding: This was a design choice. The cancellation guard is a live invariant: the hook must not operate while cancellation is enabled, because that would reopen the request-then-cancel bypass of the Reg-S transfer lock. Re-enabling cancellation fail-closes transfer, withdraw, and redeem until it is disabled again via the vault's `configure()`. That halt is the intended recovery; there is no hook-level override. We updated the NatSpec/spec to make this explicit.

HAL-015 SOLVED

LTV divergence due to missing single collateral enforcement on Dolomite helpers

• Informational · 1.3 `AO:S/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:M/Y:M`

DESCRIPTION

The `PositionHelperBase` contract gates `deploy`, `retract`, `selfLiquidate`, and `selfLiquidateToLtv` behind the `onlySingleCollateralPosition` modifier, which calls `_requireSingleCollateralPosition()`. `PositionHelperAaveV3` overrides that hook to assert the owner's account holds only the configured collateral and debt reserves, because its LTV math combines an account-wide current LTV with single reserve thresholds that agree only under that single collateral assumption.

However, neither `PositionHelperDolomiteBase` nor its concrete flavours override `_requireSingleCollateralPosition()`, so it remains the base no-op for every Dolomite helper. Meanwhile `DolomitePositionViewLib.getCurrentLtvInWad()` prices every asset in the sub-account via `getAccountValues`, while `PositionHelperDolomiteBase.getCollateralAmount()` and `getDebtAmount()`, used to size borrow and withdrawal amounts in deploy, retract, and self liquidation, read only the configured collateral and borrow markets. The Dolomite sub-account defaults to `ACCOUNT_NUMBER` 0, the owner's default public account, which commonly holds other tokens.

When the account holds any asset beyond the configured collateral and borrow pair, the account-wide LTV and liquidation threshold used by the `canLiquidate` trigger and by LTV band validation diverge from the single market amounts used to size operations, with nothing enforcing the single collateral invariant. With the sub-account holding an extra unrelated asset, the account-wide LTV and the per market sizing LTV diverge while a `deploy` call guarded by `onlySingleCollateralPosition` still executes with no revert. The self liquidation trigger can then be evaluated against an understated or overstated risk measurement, and deploy or retract sizing operates on inconsistent numbers.

RECOMMENDATION

Compute the account-wide LTV in `PositionHelperDolomiteBase` strictly from the configured collateral and borrow markets, reusing the existing `getCollateralAmount()`, `getDebtAmount()`, and `collateralPriceInBorrowTokensInRay()` helpers, so the risk measurement used by the `canLiquidate` trigger and LTV band validation stays consistent with the per market amounts that size `deploy()`, `retract()`, and self liquidation. Alternatively, override `_requireSingleCollateralPosition()` to assert the sub-account holds only the collateral and borrow markets, which requires first extending the `IDolomiteMargin` interface with the account market enumeration functions, since the current interface cannot list a sub-account's non zero markets.

REMEDIATION COMMENT

Addressed in: `064e6e1af5809b6cf2ad912976611d1762ad4516`

SOLVED: The `Blueprint Finance` **team** solved this finding in the specified commit by overriding `_requireSingleCollateralPosition()` in `PositionHelperDolomiteBase` to assert the sub-account holds only the configured collateral and borrow markets, backed by a new `IDolomiteMargin.isSingleCollateralAndDebtMarket` enumeration.

HAL-016 SOLVED

Privilege reductions lack advance notice due to asymmetric timelock

Informational · 1.0 AO:S/AC:L/AX:L/R:P/S:U/C:N/A:N/I:N/D:N/Y:C

DESCRIPTION

The `HookContainer.removeHook()`, `RedemptionClaimToken.removeFromWhitelist()`, and the `revokeRole()` and `renounceRole()` functions in `HookContainer`, `AssetCX`, `ConcreteMerchant`, `RedemptionClaimToken`, and `DolomiteLiquidationModule` all perform privilege reducing operations. The protocol uses a timelock to give vault participants advance notice of privileged changes, so additive operations such as `addHook()`, `replaceHook()`, `addToWhitelist()`, and `grantRole()` are gated by the `afterDelay` modifier or an `_enforceDelay()` call and take effect only after a matured proposal.

However, the corresponding restrictive operations execute instantly with no delay, so the advance notice guarantee applies in one direction only. This asymmetry is intentional: additive changes are delayed to give participants time to react, while restrictive corrections such as removing a hook, evicting a whitelist entry, or revoking a role are deliberately immediate so that a compromised or misbehaving component can be shut off in a single transaction.

A holder of the relevant manager or admin role can therefore strip a protective hook, a whitelist entry, or an operational role at once, temporarily disrupting the affected function set. Removing a `DepositLockHook` or `DepositLockRegSHook` lifts `preTransfer`, `preWithdraw`, and `preRedeem` enforcement; evicting the `DolomiteLiquidationModule` or `DolomiteMargin` endpoint makes subsequent `mint()`, `burn()`, and collateral supply calls revert inside `_update()`; and revoking `MERCHANT_ROLE`, `LIQUIDATION_MODULE_ROLE`, or `LIQUIDATOR_ROLE` halts minting, burning, or new liquidations. Every reduction has an additive inverse that the same trusted role can re-invoke, so no state is permanently bricked, restoring the removed entry is gated only by the standard timelock delay, and the existing last admin guard prevents an admin from locking out all administrators.

RECOMMENDATION

Keep `HookContainer.removeHook()`, `RedemptionClaimToken.removeFromWhitelist()`, and the `revokeRole()` and `renounceRole()` paths instant, and document this single direction delay as an intentional emergency kill switch. Restrict any future advance notice mechanism to additive grants only, so removing a hook, evicting a whitelist entry, or revoking a role stays immediate while the existing last admin guard continues to keep at least one administrator in place.

REMEDIATION COMMENT

Addressed in: 6f35e83562f02b899a6b86ea9b5e34c52408b4a2

SOLVED: The `Blueprint Finance` **team** solved this finding in the specified commit by reclassifying `removeHook()` as functionally additive and gating it behind `afterDelay` at TIER_2, while documenting the remaining instant restrictive operations (`revokeRole` / `renounceRole`, `removeFromWhitelist`) as an intentional kill-switch, stating:

> We have re-assessed the audit finding on privilege-reduction asymmetry: `removeHook()` was reclassified as functionally additive (in most cases it lifts hook enforcement rather than shutting off a bad actor) and is now gated behind `afterDelay` at TIER_2, wired into both `initialize` and `reinitializeV2`. The remaining instant restrictive corrections (`revokeRole` / `renounceRole`, `removeFromWhitelist`, etc.) were left as-is and their intentional kill-switch design was documented in NatSpec and `doc/spec/Timelock.md`.

HAL-017 SOLVED

Block of hook and PMH cloning due to missing unblock functions

Informational · 1.0 AO:S/AC:L/AX:L/R:P/S:U/C:N/A:C/I:N/D:N/Y:N

DESCRIPTION

The `PeripheryFactory` contract gates cloning of approved implementations behind an approval flag and a block flag keyed by implementation ID, so that a flagged implementation can no longer be cloned. For strategy implementations the lifecycle is symmetric: `PeripheryFactory.approveStrategyImpl()` and `PeripheryFactory.blockStrategyImpl()` are callable by the factory manager, and `PeripheryFactory.unblockStrategyImpl()` is callable by the owner after the timelock delay, giving the higher trust owner a governed path to reverse a block.

However, the hook and PMH lifecycles are asymmetric. `PeripheryFactory.approveHookImpl()` / `blockHookImpl()` and `PeripheryFactory.approvePMHImpl()` / `blockPMHImpl()` exist, but there is no `unblockHookImpl` or `unblockPMHImpl` anywhere in the contract. `blockHookImpl()` and `blockPMHImpl()` are callable by the factory manager with no delay and set the blocked flag for the implementation ID to true, after which `PeripheryFactory.cloneHook()` / `clonePMH()` revert with `ImplBlocked` on every subsequent call. Re-approval of the same implementation ID is impossible because the implementation is still recorded and the approval functions revert with `ImplAlreadyApproved`.

The factory manager is documented as a lower trust, rotatable operational role, yet it can disable cloning of any hook or PMH implementation ID indefinitely. The factory is UUPS upgradeable and `_authorizeUpgrade` is restricted to the owner after the timelock delay, so the owner can still reverse a block by upgrading the implementation. No dedicated unblock function exists, though, so reversal today requires a full factory upgrade or the deployment and approval of a brand new implementation version to obtain a fresh implementation ID. Existing clones are unaffected.

RECOMMENDATION

Add `unblockHookImpl` and `unblockPMHImpl` to `PeripheryFactory`, callable by the owner after the timelock delay, that clear the blocked flag for the given implementation ID and emit a matching unblock event, mirroring `PeripheryFactory.unblockStrategyImpl()`, while keeping `blockHookImpl()` and `blockPMHImpl()` instant and manager callable.

Additionally, register both new selectors as TIER_2 artifacts in `TimelockConfigurationLib.seedPeripheryFactoryArtifacts` alongside the existing `unblockStrategyImpl` entry, and add them through `Timelock.addArtifact` on already deployed timelocks, so `propose()` accepts them once the timelock is activated and the unblock path stays callable.

REMEDIATION COMMENT

Addressed in: 3403ac893e956331d450d22681bcd74df3cc592b

SOLVED: The `Blueprint Finance` **team** solved this finding in the specified commit by adding owner-only, timelocked `unblockHookImpl` and `unblockPMHImpl` functions to `PeripheryFactory` and registering both selectors as TIER_2 timelock artifacts.

HAL-018 ACKNOWLEDGED

Self-liquidation gate ignores in-flight liquidation exposure in PositionHelperDolomiteAssetCX

• Informational · 1.0 AO:S/AC:L/AX:L/R:N/S:U/C:N/A:N/I:M/D:N/Y:N

DESCRIPTION

The `PositionHelperDolomiteAssetCX` contract manages an AssetCX collateralized leverage position whose self-liquidation is delegated to a dedicated liquidation module. During a liquidation, debt is moved onto a module owned temporary Dolomite account and the primary collateral is redeemed out, so the position's true risk is split between the primary sub-account and those temporary positions. To capture this, the contract exposes `PositionHelperDolomiteAssetCX.getCurrentLtvWithLiquidationsInWad()`, which folds the collateral and debt sitting in the module's in-flight temporary positions into the LTV, alongside `getPosition()`, `getCollateralAmount()`, and `getDebtAmount()`, which it overrides to report the primary sub-account only.

However, `PositionHelperDolomiteAssetCX` does not override `getCurrentLtvInWad()`, so the `canLiquidate` trigger inherited from `PositionHelperBase` and the `canTransfer` guard both evaluate the primary sub-account LTV and ignore the in-flight liquidation exposure. The liquidation aware view `PositionHelperDolomiteAssetCX.getCurrentLtvWithLiquidationsInWad()` is not consulted by any on-chain guard.

While a liquidation is in flight, the gate LTV can diverge materially from the position's true aggregate LTV. When the aggregate LTV is above the liquidation threshold but the primary only LTV still reads healthy, a `POSITION_OPERATOR_ROLE` holder is wrongly blocked from further self-liquidation, leaving a liveness gap on the recovery path that only `POSITION_ADMIN_ROLE` can work around. Conversely, a primary only reading that looks safe can permit collateral transfers out of the tracked account while unsettled debt sits in the temporary account, with Dolomite's own post-action solvency check on the primary account remaining the only backstop against outright insolvency.

RECOMMENDATION

Mark `getCurrentLtvInWad()` virtual in `PositionHelperDolomiteBase` and override it in `PositionHelperDolomiteAssetCX` to return the aggregate value produced by `getCurrentLtvWithLiquidationsInWad()`, so the inherited self liquidation trigger evaluates the full in flight exposure. Additionally, apply this aggregate liquidation aware LTV in the `transferFromDefaultAccount()` path of `PositionHelperDolomiteAssetCX`, where the source is `ACCOUNT_NUMBER`, and keep `transferToDefaultAccount()` evaluating the per account LTV of its arbitrary source via `_getCurrentLtvInWad(sourceAccountNr)`. Additionally, value the redemption claim leg in `getCurrentLtvWithLiquidationsInWad()` at its oracle price, or, if it is valued at par, scale it by `collateralRatioWad`, so every gate acts on an accurate aggregate LTV.

REMEDIATION COMMENT

ACKNOWLEDGED: The `Blueprint Finance` **team** made a business decision to acknowledge this finding and not alter the contracts, stating:

> We only track the main account's ltv for operator interventions, not the aggregate. The aggregate is just for the integrations to understand the total position, but has no implication on triggerability by an operator. Also this scenario: While a liquidation is in flight, the gate LTV can diverge materially from the position's true aggregate LTV. When the aggregate LTV is above the liquidation threshold but the primary only LTV still reads healthy, a `POSITION_OPERATOR_ROLE` holder is wrongly blocked from further self-liquidation, is not true, if anything the aggregate ltv is worse and the operator would not be blocked.

HAL-019 SOLVED

Denial of service due to missing zero price guard in DolomitePositionViewLib.collateralPriceInBorrowTokensInRay

• Informational · 0.7 AO:S/AC:L/AX:H/R:N/S:U/C:N/A:C/I:N/D:N/Y:N

DESCRIPTION

The `DolomitePositionViewLib.collateralPriceInBorrowTokensInRay()` function is a shared view helper used by the Dolomite position helpers to express the price of one unit of collateral in borrow token terms. It reads two Dolomite oracle prices through `dolomiteMargin.getMarketPrice()`, one for the collateral market and one for the borrow market, and returns `cPrice.mulDiv(RAY, bPrice)`. This value feeds the self-liquidation, deploy, and retract math across every Dolomite flavour.

However, the `DolomitePositionViewLib.collateralPriceInBorrowTokensInRay()` function does not validate that either price is non zero before dividing. The equivalent helpers for the other supported lenders, `PositionHelperAaveV3.collateralPriceInBorrowTokensInRay()` and `PositionHelperMorpho.collateralPriceInBorrowTokensInRay()`, both revert with a descriptive error when a price is zero, so the Dolomite path is the only one lacking the guard.

When Dolomite's oracle returns zero for a market (for example a paused, migrated, stale, or misconfigured feed), two failure modes result. If the borrow market price is zero, the `mulDiv` divides by zero and reverts with an opaque `Panic(0x12)` in every caller. If the collateral market price is zero, the function silently returns zero, and that value propagates into `PositionHelperDolomiteNative.selfLiquidateDolomiteNative()`, where `debtAmount.mulDiv(RAY, 0, Math.Rounding.Ceil)` triggers a second division by zero panic.

As a result, during an oracle outage the time sensitive self-liquidation entry points `PositionHelperDolomiteNative.selfLiquidateDolomiteNative()` and `PositionHelperDolomiteNative.selfLiquidateDolomiteNativeToLtv()` revert with an uninformative panic rather than a descriptive error, and a position approaching the Dolomite liquidation threshold cannot be unwound through self-liquidation until the feed recovers.

RECOMMENDATION

In `DolomitePositionViewLib.collateralPriceInBorrowTokensInRay()`, revert with a descriptive named error such as `ZeroMarketPrice(marketId)` when either the collateral price or the borrow price is zero, before performing the `mulDiv`, matching the zero price guards already present in `PositionHelperAaveV3.collateralPriceInBorrowTokensInRay()` and `PositionHelperMorpho.collateralPriceInBorrowTokensInRay()`.

REMEDIATION COMMENT

Addressed in: ebd8a3e3b705f3c371a804fc1d04f3cf933a7bec

SOLVED: The `Blueprint Finance` **team** solved this finding in the specified commit by reverting `ZeroMarketPrice(marketId)` when either the collateral or borrow oracle price is zero, before the division.

HAL-020 SOLVED

Arithmetic panic at extreme LTV due to unclamped uint64 LTV handling

Informational · 0.6 AO:A/AC:L/AX:L/R:F/S:U/C:N/A:N/I:L/D:N/Y:N

DESCRIPTION

The Dolomite and Aave position helpers express a position's loan to value ratio in WAD as a uint64, both when narrowing a computed ratio with SafeCast toUint64() and when adding two uint64 LTV terms. The maximum representable uint64 in WAD terms is about 18.44e18, which corresponds to an LTV of roughly 1845%.

However, several of these paths perform the narrowing cast or the addition without first widening to uint256 or clamping to the uint64 ceiling. In Solidity 0.8 both a SafeCast overflow and a uint64 + uint64 overflow revert with an opaque arithmetic panic instead of returning the intended value or the descriptive named error. The sibling implementations `PositionHelperMorpho.getCurrentLtvInWad()` and `PositionHelperBase.canLiquidate()` already guard the same operation, the former by clamping with Math.min(ltv, type(uint64).max) and the latter by widening to uint256 before the addition, so the affected paths are the only ones missing the guard.

Because getCurrentLtvInWad() feeds the canLiquidate modifier that gates operator triggered self liquidation, and canTransfer gates operator initiated sub account transfers, a position that is deeply underwater beyond roughly 1845% LTV causes these views to revert with an uninformative panic at the moment self liquidation or a denial is most needed, rather than returning the uint64 ceiling or reverting with OperatorNotAllowedSourceAccountInSelfLiqZone. The POSITION_ADMIN_ROLE branch of canLiquidate is evaluated first and remains a backstop, the operator is correctly denied at these LTVs, and the configured self liquidation band forces the trigger far below 1845%, so the reverting state is only reachable under an extreme single block price dislocation.

Instances of this issue include:

- `PositionHelperAaveV3.getCurrentLtvInWad()`, `LoopingPrimitiveLib._currentLtvInWad()`, `DolomitePositionViewLib.getCurrentLtvInWad()`, `PositionHelperDolomiteAssetCX.getCurrentLtvWithLiquidationsInWad()`, and `AaveV3LenderAdapter.getCurrentLtvInWad()`: narrow debt.mulDiv(WAD, collateral) with SafeCast toUint64() and apply no clamp, so an LTV above the uint64 ceiling reverts with an overflow panic.
- `PositionHelperDolomiteBase.canTransfer()`: computes _getCurrentLtvInWad(sourceAccountNr) + selfLiqThrBufferInWad with both operands as uint64, so an LTV within selfLiqThrBufferInWad of type(uint64).max reverts with an addition overflow panic instead of the intended OperatorNotAllowedSourceAccountInSelfLiqZone revert.

RECOMMENDATION

Apply the Math.min(ltv, type(uint64).max) clamp before the toUint64() cast in `PositionHelperAaveV3.getCurrentLtvInWad()`, `LoopingPrimitiveLib._currentLtvInWad()`, `DolomitePositionViewLib.getCurrentLtvInWad()`,

`PositionHelperDolomiteAssetCX.getCurrentLtvWithLiquidationsInWad()`, and `AaveV3LenderAdapter.getCurrentLtvInWad()`, matching `PositionHelperMorpho.getCurrentLtvInWad()`.

Additionally, widen both operands to uint256 before the comparison in `PositionHelperDolomiteBase.canTransfer()`, matching `PositionHelperBase.canLiquidate()`.

REMEDIATION COMMENT

Addressed in: `064e6e1af5809b6cf2ad912976611d1762ad4516`

SOLVED: The `Blueprint Finance` **team** solved this finding in the specified commit by clamping every LTV narrowing with `Math.min(ltv, type(uint64).max)` before the `toUint64()` cast and widening the `canTransfer` operands to `uint256`.

HAL-021 SOLVED

LTV divergence due to missing decimal conversion of redemption claim collateral

Informational · 0.6 AO:A/AC:L/AX:L/R:F/S:U/C:N/A:N/I:L/D:N/Y:N

DESCRIPTION

The `PositionHelperDolomiteAssetCX.getCurrentLtvWithLiquidationsInWad()` function is a view that reports a position's loan to value (LTV) including any in-flight liquidation's temporary position. It sums the primary collateral, converted to borrow token terms via `collateralPriceInBorrowTokensInRay()`, with the redemption claim collateral amount, then divides total debt by that combined collateral value. The redemption claim leg is added as a raw token amount on the assumption, stated in the code comment, that a `RedemptionClaimToken` is worth the borrow token one to one at par.

However, the redemption claim amount is added in its own base units with no decimal conversion, while the other collateral and debt legs are expressed in borrow token units. The `RedemptionClaimToken` decimals are configurable at initialization and need not equal the borrow token's decimals, so the two legs can be summed on incompatible scales. When the decimals differ, the combined collateral value diverges from its true borrow token value by orders of magnitude, skewing the reported LTV in either direction depending on which token has more decimals: understated when the claim token has more decimals than the borrow token, and overstated when it has fewer.

Because this view is not consumed by any on-chain gate, since the transfer and self liquidation checks use the primary position LTV, the impact is confined to off-chain consumers such as monitoring and the LTV helper integration, which may read an understated LTV and mistime liquidation decisions.

RECOMMENDATION

In `PositionHelperDolomiteAssetCX.getCurrentLtvWithLiquidationsInWad()`, normalize the redemption claim amount to borrow token units before adding it to the priced primary collateral, scaling it by `10**(borrowDecimals - rcDecimals)` (multiply when the borrow token has more decimals, divide otherwise) so that every collateral and debt leg shares a single unit basis.

REMEDIATION COMMENT

Addressed in: fef265262badf15e1196098fb5fa34a554002bfb

SOLVED: The `Blueprint Finance` **team** solved this finding in the specified commit by normalizing the redemption-claim collateral leg from its own token decimals to borrow-token decimals before summing it into the combined collateral value.

HAL-022 ACKNOWLEDGED

Owner blocked from emergency access before strategy configuration due to check ordering in modifier

Informational · 0.5 AO:S/AC:L/AX:L/R:F/S:U/C:N/A:C/I:N/D:N/Y:N

DESCRIPTION

The `AaveV3LenderAdapter.supply()` and `AaveV3LenderAdapter.repay()` functions are guarded by the `onlyStrategyOrOwner` modifier, which is intended to let both the configured strategy and the contract owner act as an emergency fallback. The modifier first reads the stored strategy address and reverts with `ZeroStrategyAddress` when it is `address(0)`, before it ever evaluates whether the caller is the owner.

However, the strategy field defaults to `address(0)` until `setStrategy()` is called. During that bootstrap window the zero-address check fires for every caller, including the owner, so the owner cannot call `supply()` or `repay()` even though the modifier's name promises owner access. This bootstrap-window logic in `onlyStrategyOrOwner`, `supply()`, and `repay()` is unmodified by this change: only a blank-line insertion touched the file.

The exposure is bounded and reversible. The owner can end the window immediately by calling `setStrategy()` to install a non-zero strategy and unblock `supply()` and `repay()`, and `rescueToken()` (`onlyOwner`) can recover any tokens sent to the adapter during the window. Because no position exists before a strategy is set, the availability impact is a brief inconsistency between the modifier's intent and its behavior rather than a true owner lockout.

RECOMMENDATION

In `AaveV3LenderAdapter.onlyStrategyOrOwner`, evaluate the owner branch before the zero-address check so the owner can always call `supply()` and `repay()` regardless of whether a strategy has been configured.

REMEDIATION COMMENT

ACKNOWLEDGED: The `Blueprint Finance` **team** made a business decision to acknowledge this finding and not alter the contracts, stating:

> Before `setStrategy()` is called there is no position, collateral and debt are both zero, so there is nothing for the owner's emergency access to protect. That access exists to defend a live position when the strategy is paused or broken, which cannot happen before a strategy exists.

HAL-023 SOLVED

Forced self liquidation due to unchecked destination account in debt transfers

Informational · 0.5 AO:S/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:L/Y:N

DESCRIPTION

The `PositionHelperDolomiteBase.transferToDefaultAccount()` and `PositionHelperDolomiteBase.transferFromDefaultAccount()` functions move a chosen market balance between two of the position owner's Dolomite sub-accounts, gated by the `canTransfer` modifier. That modifier lets a `POSITION_OPERATOR_ROLE` holder proceed only when the source account's LTV plus `selfLiqThrBufferInWad` stays below the liquidation threshold, so an operator cannot manipulate an account that is already in the self liquidation danger zone.

However, `canTransfer` validates only the source account, and for a debt balance in the borrow market the source is the account that gets safer, not riskier. `DolomiteAccountActionLib.transfer()` uses a Dolomite `Transfer` action where the counterparty absorbs the moved balance, so moving debt out of a sub-account improves that source account while loading the debt onto the destination. When the destination is the main `ACCOUNT_NUMBER` position, the guard evaluates the wrong account: it checks the debt shedding source, which trivially passes, and never checks the destination whose LTV actually rises. Dolomite enforces only its own hard solvency threshold at the end of the operation, not the helper's buffer.

An operator can therefore move debt into the main position and drive its LTV into the `[liquidationThreshold − selfLiqThrBufferInWad, liquidationThreshold)` band that the buffer exists to keep operators out of, then invoke self liquidation, which is now permitted, forcing a slippage bearing unwind on the position owner. Moving the secondary account's borrow market debt onto the main account drives the main LTV from 0 into that band while the source stays healthy and the guard passes, after which the operator self liquidation call clears the `canLiquidate` gate that would otherwise have rejected it.

RECOMMENDATION

In the `canTransfer` modifier of `PositionHelperDolomiteBase`, when the moved market is the borrow market, evaluate the safety of the destination account that gains debt, or validate both endpoints against `liquidationThreshold − selfLiqThrBufferInWad` after the transfer.

REMEDIATION COMMENT

Addressed in: 064e6e1af5809b6cf2ad912976611d1762ad4516

SOLVED: The `Blueprint Finance` **team** solved this finding in the specified commit by validating both transfer endpoints post-transfer via `_requireTransferLeftAccountsSafe`, so the destination account that gains debt is checked, not only the source.

HAL-024 SOLVED

Oversized position unwind due to unclamped debt amount in selfLiquidateDolomiteNative

Informational · 0.5 AO:S/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:L/Y:N

DESCRIPTION

The `PositionHelperDolomiteNative.selfLiquidateDolomiteNative()` function performs a Dolomite native self-liquidation by selling collateral for the borrow token and crediting the proceeds against the position's debt. It sizes the collateral to sell as `collateralAmountToSell = debtAmount.mulDiv(RAY, collateralPriceInBorrowTokensInRay(), Ceil)`, deriving the amount directly from the caller supplied `debtAmount`. It is reachable by a `POSITION_OPERATOR_ROLE` holder once `canLiquidate` admits the caller, namely once the position LTV plus the self-liquidation buffer reaches the liquidation threshold.

However, `PositionHelperDolomiteNative.selfLiquidateDolomiteNative()` never clamps `debtAmount` to the position's actual outstanding debt returned by `getDebtAmount()`, whereas the sibling `PositionHelperDolomiteNative.selfLiquidateDolomiteNativeToLtv()` does clamp its computed repayment down to the current debt before sizing the sale. An operator can therefore pass a `debtAmount` larger than the debt actually owed.

When `debtAmount` exceeds the real debt, the sale is sized off the inflated figure, so more collateral is sold than is required to clear the position. The borrow token proceeds beyond the debt do not repay anything and are instead credited as a positive borrow market supply balance, so the position is unwound further than requested and a larger slice of the owner's collateral is routed through the native swap. Value on the excess is roughly preserved net of swap costs, and the resulting harm is an oversized, unrequested unwind of the owner's position executed by a semi-trusted operator.

RECOMMENDATION

In `PositionHelperDolomiteNative.selfLiquidateDolomiteNative()`, clamp `debtAmount` to `getDebtAmount()` before computing `collateralAmountToSell`, matching the clamp already applied in `PositionHelperDolomiteNative.selfLiquidateDolomiteNativeToLtv()`.

REMEDIATION COMMENT

Addressed in: 28072663cdb1c86f84a2a0005e1435ac02d3c412

SOLVED: The `Blueprint Finance` **team** solved this finding in the specified commit by clamping the caller-supplied `debtAmount` down to the position's current debt before sizing the collateral sale.

Disclaimer

Halborn strongly recommends conducting a follow-up assessment of the project either within six months or immediately following any material changes to the codebase, whichever comes first. This approach is crucial for maintaining the project's integrity and addressing potential vulnerabilities introduced by code modifications.